WEEK 8 ASSIGNMENT REPORT

E-COMMERCE ORDER ANALYTICS SYSTEM

1. INTRODUCTION

The E-Commerce Order Analytics System is an end-to-end data analytics project developed using Python and SQL. The system generates realistic e-commerce datasets, cleans and validates the data using Pandas, stores the cleaned data in an SQLite database, and performs advanced SQL analytics to generate business insights. The project also includes a command-line reporting tool that dynamically generates reports based on user input.

2. OBJECTIVE

The objective of this project is to design and develop an end-to-end e-commerce analytics system that performs data generation, data cleaning, database loading, SQL analytics, customer segmentation, cohort analysis, and command-line reporting while ensuring data quality and robustness.

3. TECHNOLOGIES USED

Technology	Purpose
Python	Dataset generation and scripting
Pandas	Data cleaning and preprocessing
Faker	Synthetic dataset generation
SQLite	Database management
SQL	Business analytics
Tabulate	CLI report formatting
VS Code	Development environment

4. PROJECT WORKFLOW

Submitted By: Jiya Singla(MMDU)

5. IMPLEMENTATION STEPS

Step 1: Dataset Generation

Realistic e-commerce datasets were generated using Python with the **Faker** and **random** libraries. Four CSV files were created:

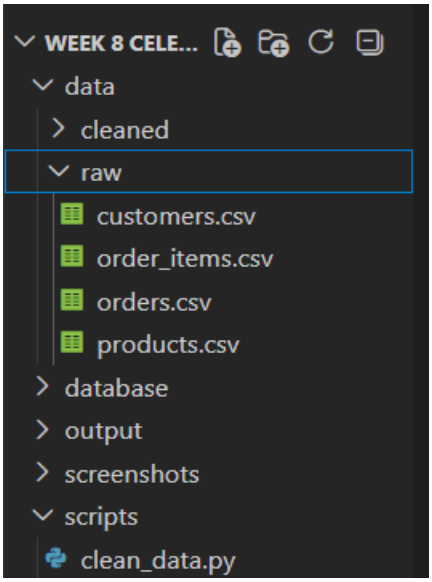
- customers.csv
- products.csv
- orders.csv
- order_items.csv

Screenshot 1: Step1_Generate_Data_Output

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS                    powershell
(venv) PS C:\Week 8 Celebal Assignment (Ecommerce Mini Project)> python scripts/generate_data.py
products.csv saved (500 records)
orders.csv saved (500 records)
order_items.csv saved (700 records)

Generation Completed Successfully


Dataset Summary
----------------------------
Customers    : 500
Products     : 500
Orders       : 500
Order Items  : 700
(venv) PS C:\Week 8 Celebal Assignment (Ecommerce Mini Project)>
```

To simulate real-world data, the datasets include missing values, duplicate records, invalid dates, incorrect email addresses, mismatched IDs, and invalid discount values. The generated files were saved in the **data/raw/** folder.

Screenshot 2: Step1_Raw_CSV_Files



Step 2: Data Cleaning and Validation

The raw datasets were loaded into Pandas DataFrames for cleaning.

The following operations were performed:

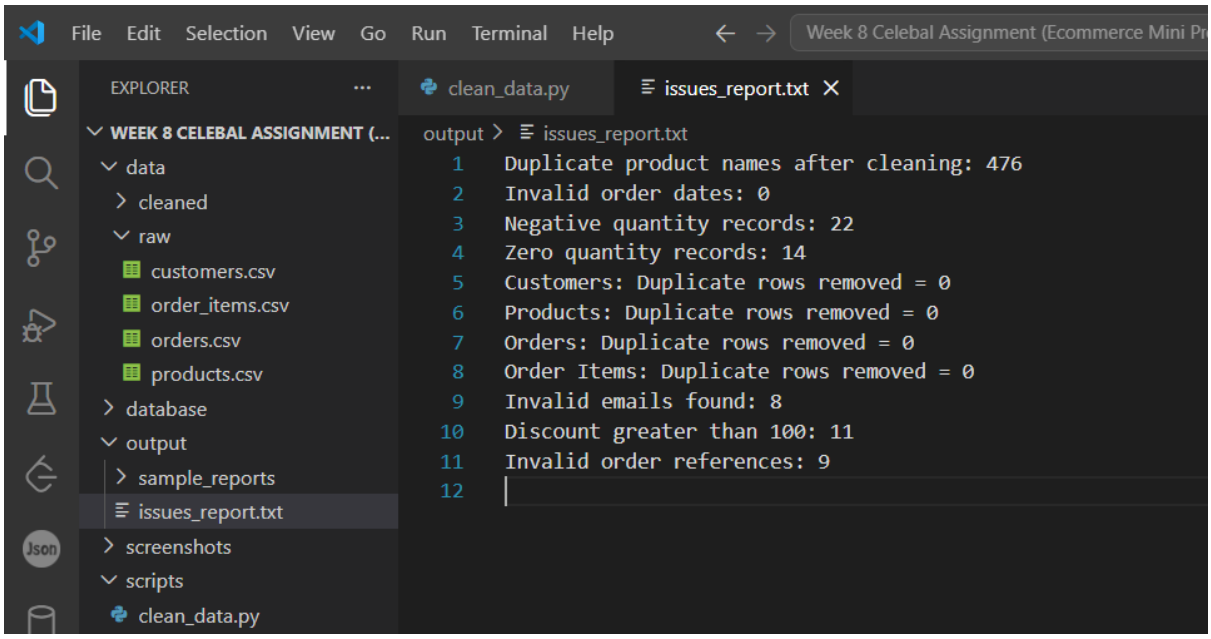
- Removed duplicate records
- Handled missing values
- Standardized date formats
- Cleaned product names
- Validated email addresses
- Checked referential integrity between tables
- Identified invalid quantities and discount values

Screenshot 3: Step2_Clean_Data_Output

```
(venv) PS C:\Week 8 Celebal Assignment (Ecommerce Mini Project)> python scripts/clean_data.py

Loading datasets...

customers_clean.csv created
products_clean.csv created
orders_clean.csv created
order_items_clean.csv created
issues_report.txt created

Invalid Email Customer IDs
[42, 138, 207, 237, 390, 391, 442, 493]

Invalid Order References
     order_item_id  order_id  product_id  quantity  unit_price  discount_percent
12              13       504         351         4     5856.96                16
191            192       507         108         4     5267.97                27
248            249       515         318         3     1505.58                 3
265            266       507         451         5     7924.06                14
362            363       520         493         3     3891.21                21
456            457       519         382         4     2207.29                40
462            463       502         395         1     7521.07               101
488            489       510         104         1      863.31                36
664            665       504         392         5     3864.91                 6

Cleaning Summary
-----------------------------------
Customers      : 500
Products       : 500
Orders         : 500
Order Items    : 700
Invalid Emails : 8
Invalid Orders : 9

Data Cleaning Completed Successfully!
(venv) PS C:\Week 8 Celebal Assignment (Ecommerce Mini Project)>
```

An issues_report.txt file was generated to record validation issues.

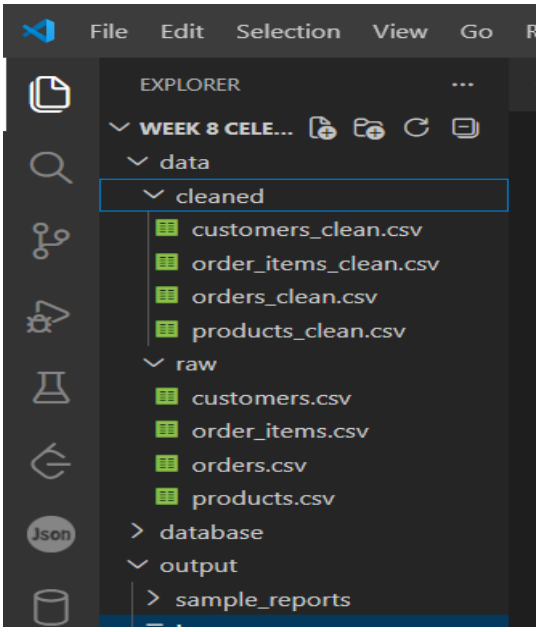
Screenshot 4: Step2_Issues_Report



The cleaned datasets were stored in the data/cleaned/ folder.

Screenshot 5: Step2_Cleaned_CSV_Files

Submitted By: Jiya Singla(MMDU)

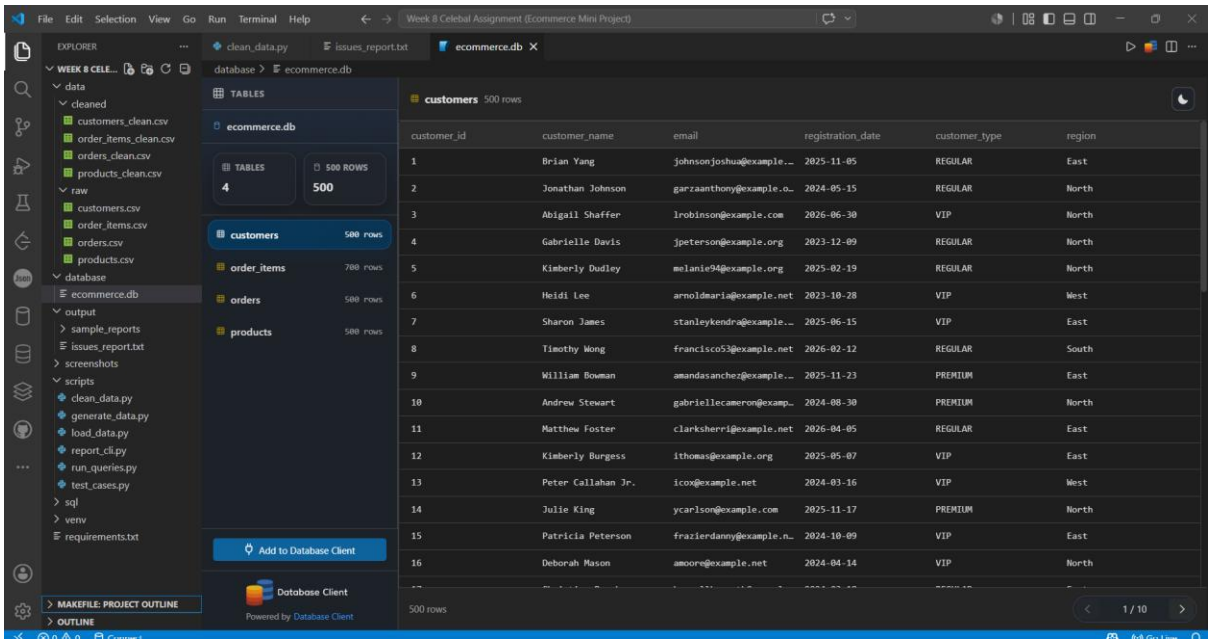


Step 3: Loading Data into SQLite

The cleaned CSV files were imported into an SQLite database.

The database schema was created using Primary Keys, Foreign Keys, and NOT NULL constraints.

Screenshot 6: Step3_SQLite_Database_Tables



Relationships between all tables were verified after loading.

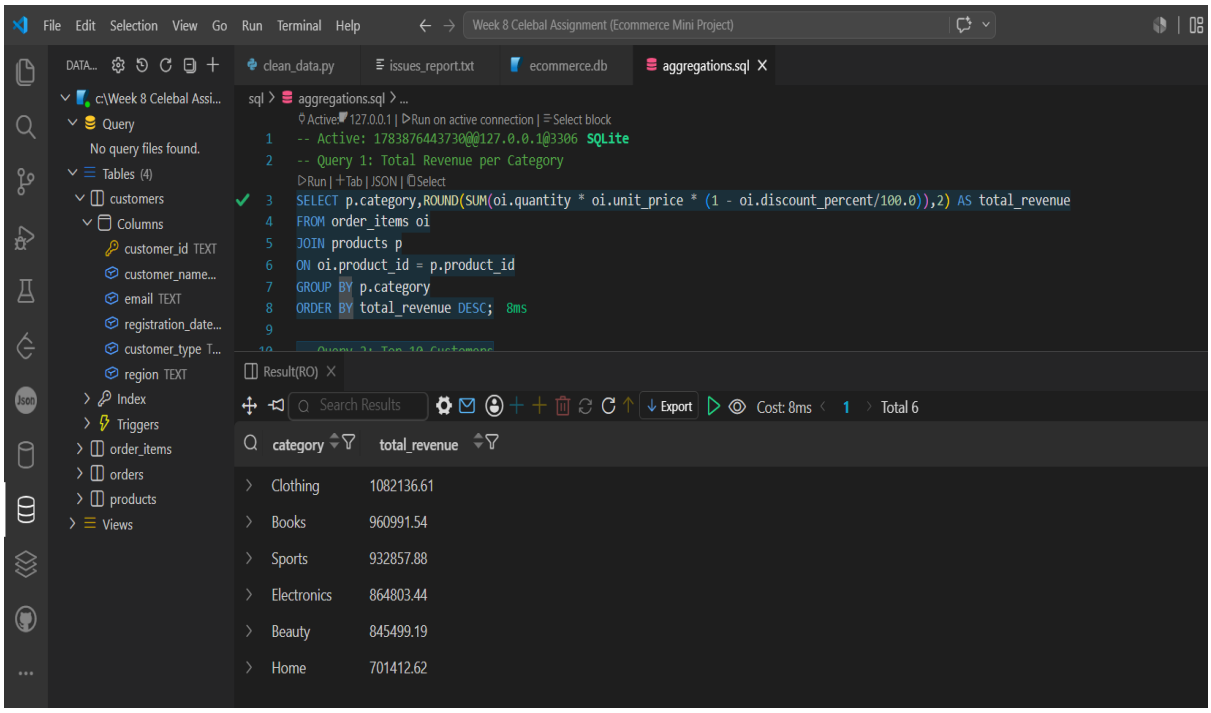
Step 4: SQL Analytics

Submitted By: Jiya Singla(MMDU)

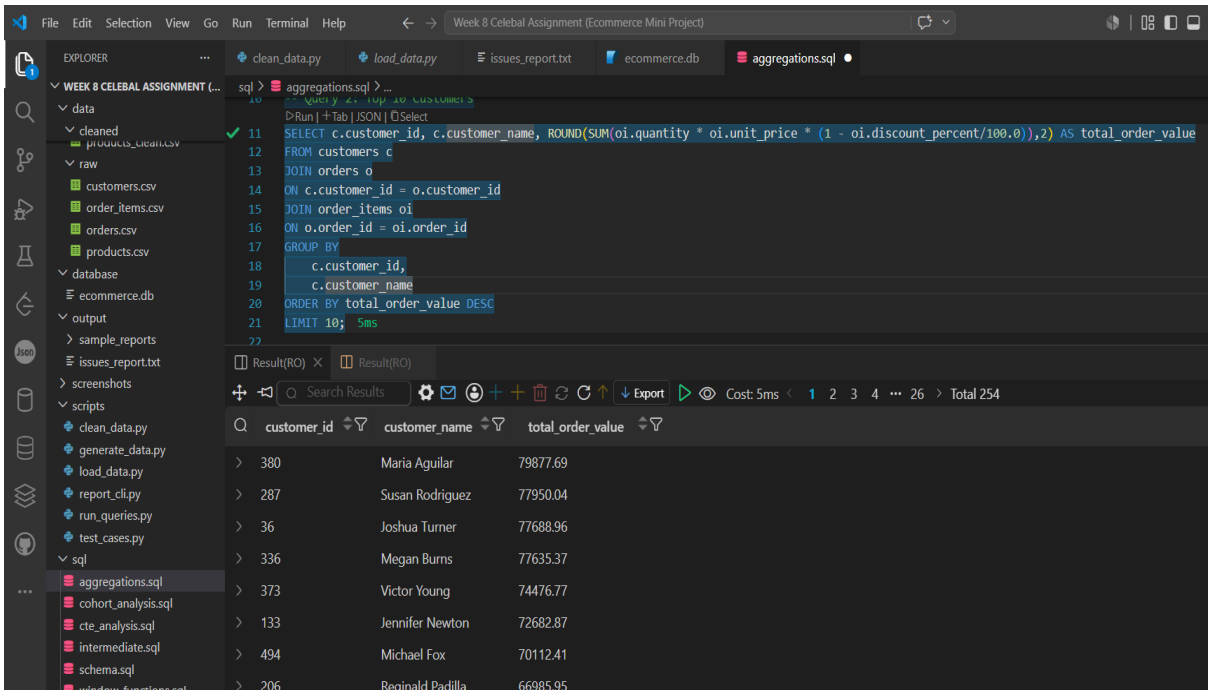
SQL queries were executed to generate business insights.

The SQL analysis includes revenue by category, top customers, monthly order trends, average order value, return rate, customer segmentation, and product performance using joins and aggregation queries.

Screenshot 7: Step4_Revenue_By_Category

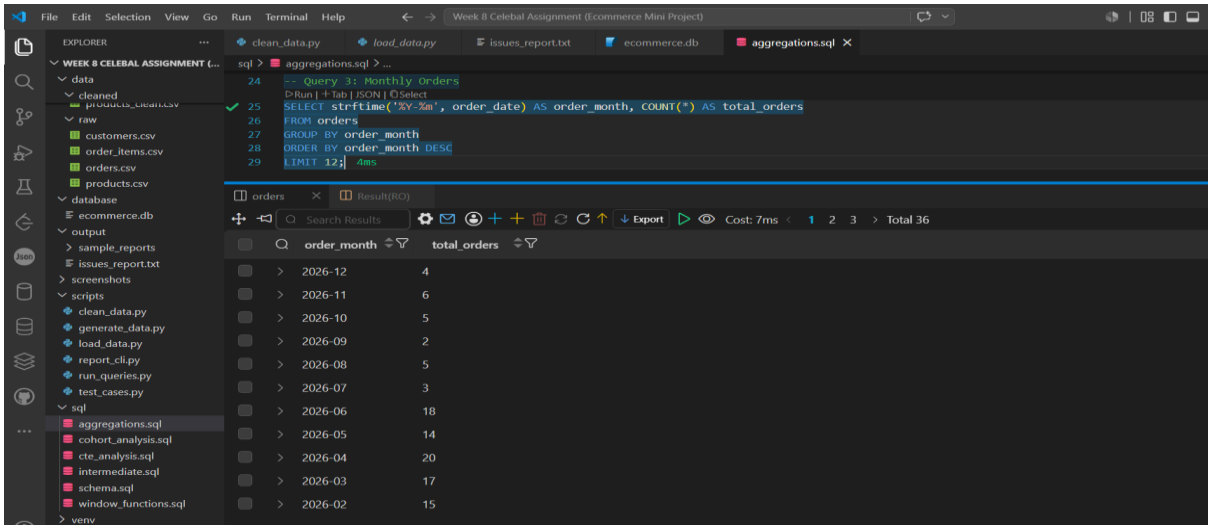


Screenshot 8: Step4_Top_Customer



Submitted By: Jiya Singla(MMDU)

Screenshot 9: Step4_Monthly_Orders

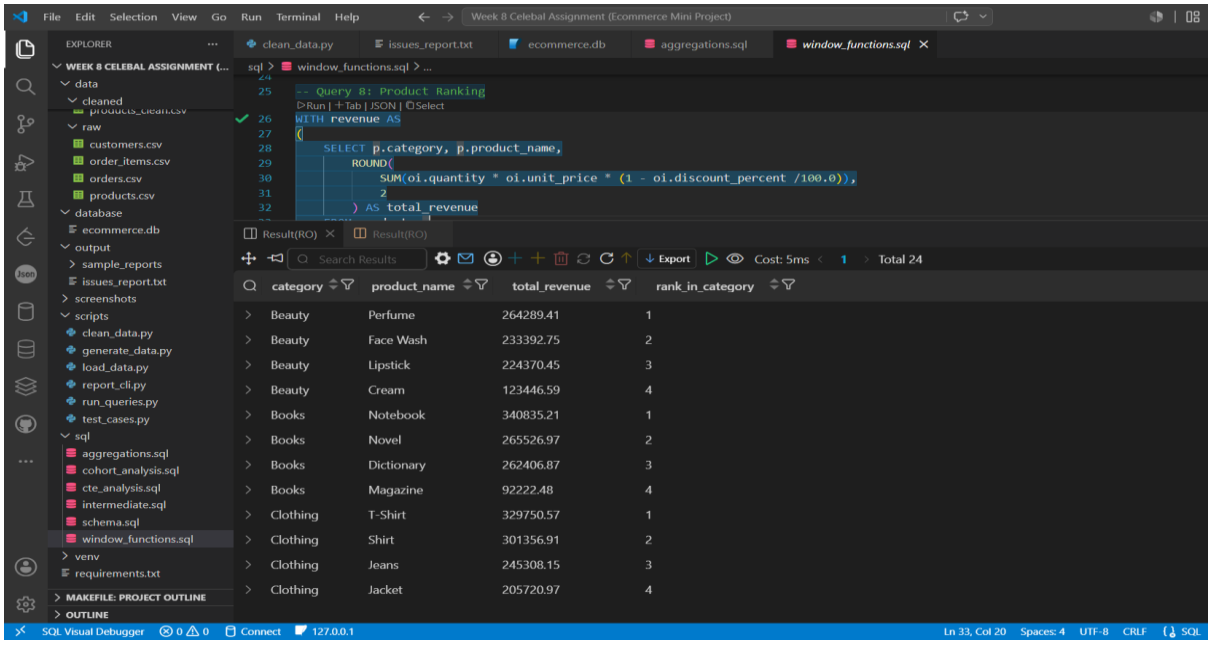


Step 5: Window Functions and CTEs

Advanced SQL analytics were implemented using Window Functions and Common Table Expressions (CTEs).

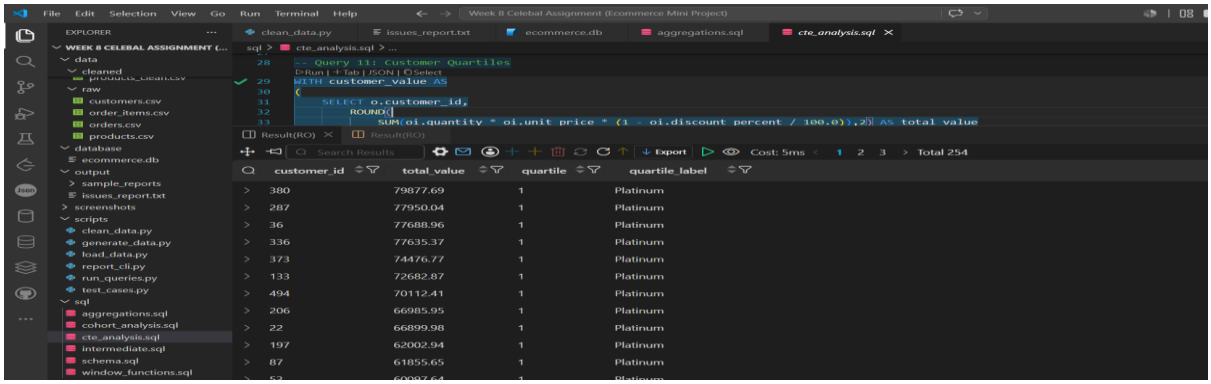
Window functions and CTEs were used to implement running totals, product ranking using DENSE_RANK(), customer order gap analysis using LAG(), revenue contribution, monthly customer segmentation, customer quartiles using NTILE(), and year-over-year revenue comparison.

Screenshot 10: Step5_Product_Ranking

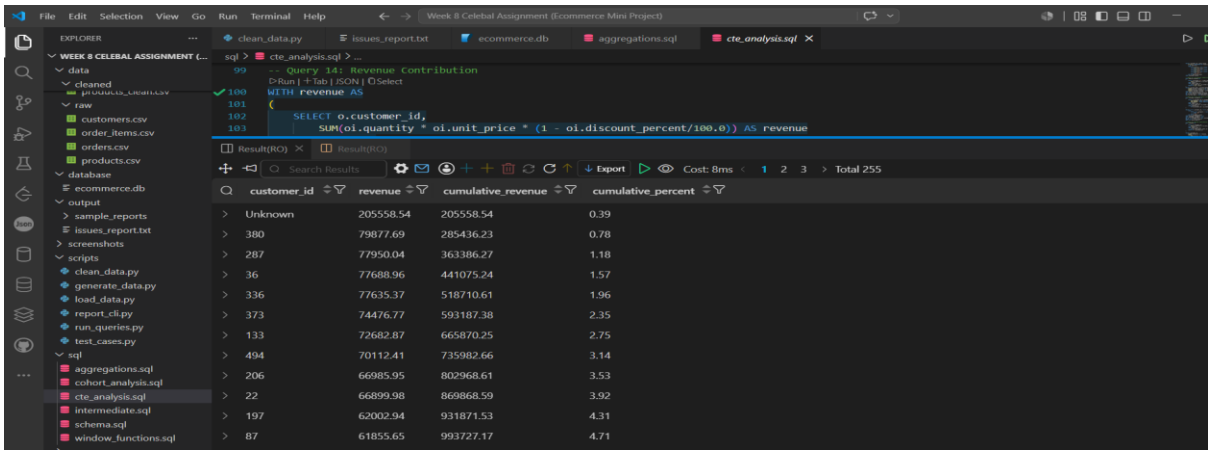


Submitted By: Jiya Singla(MMDU)

Screenshot 11: Step5_Customer_Quartiles



Screenshot 12: Step5_Revenue_Contribution

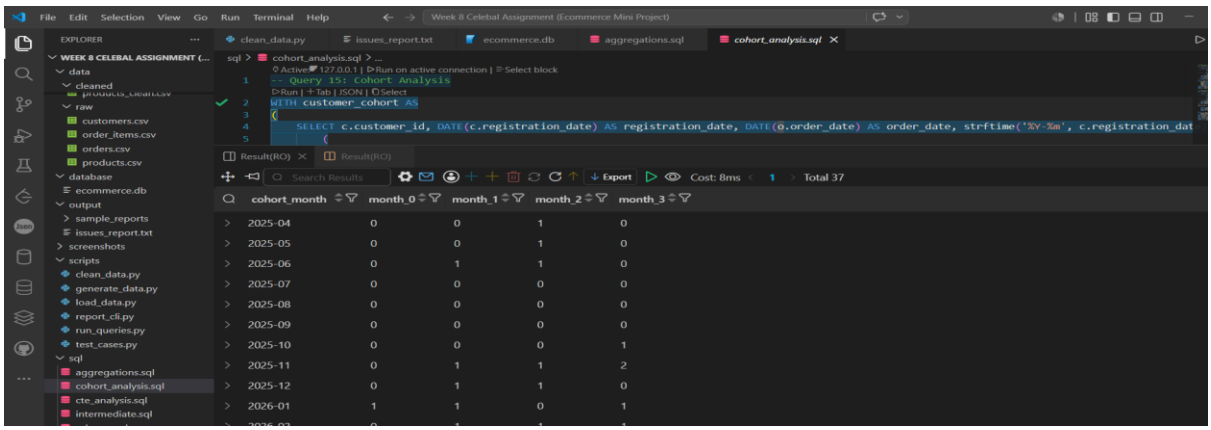


Step 6: Cohort and Retention Analysis

Customer cohorts were created based on their registration month.

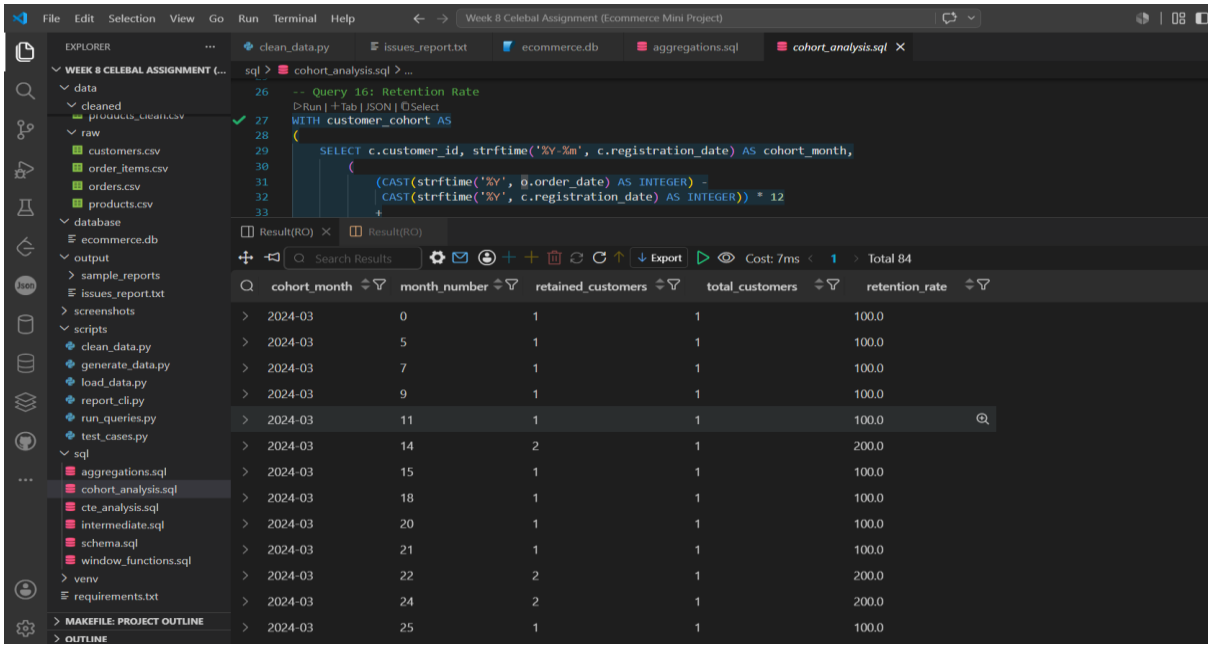
The analysis includes Cohort Analysis, Monthly Retention Rate, and Repeat Customer Tracking.

Screenshot 13: Step6_Cohort_Analysis



Submitted By: Jiya Singla(MMDU)

Screenshot 14: Step6_Retention_Rate

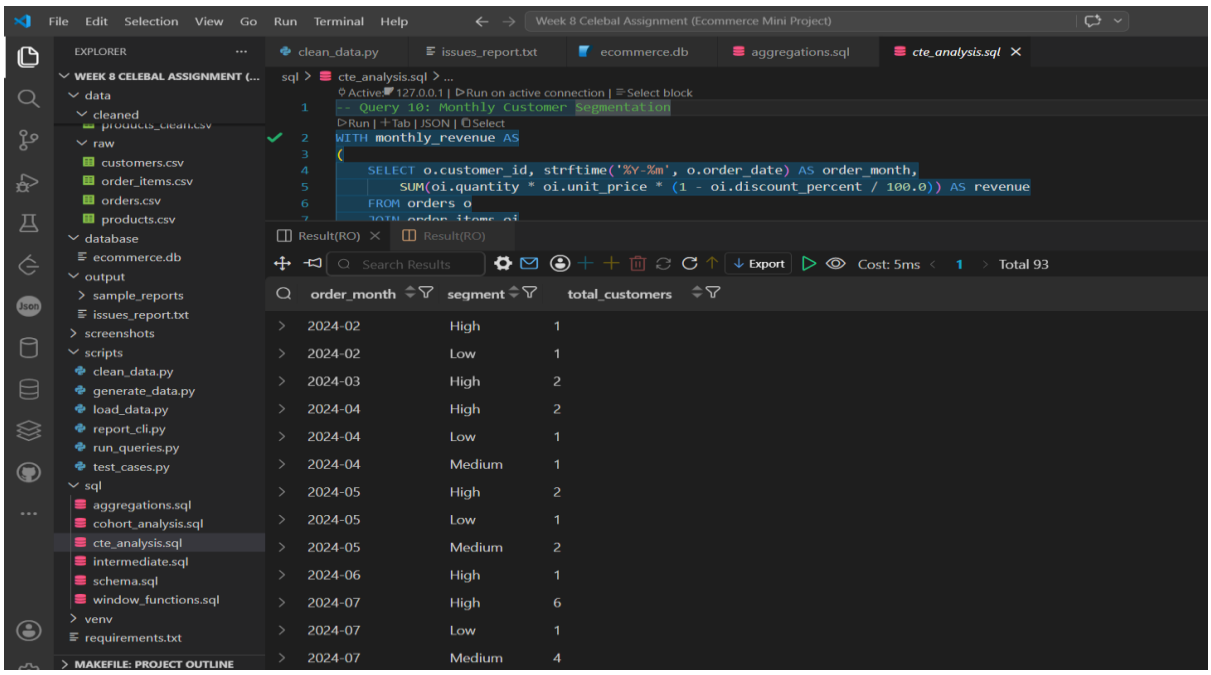


Step 7: Customer Segmentation

Customers were segmented based on purchasing behaviour using revenue and purchase frequency.

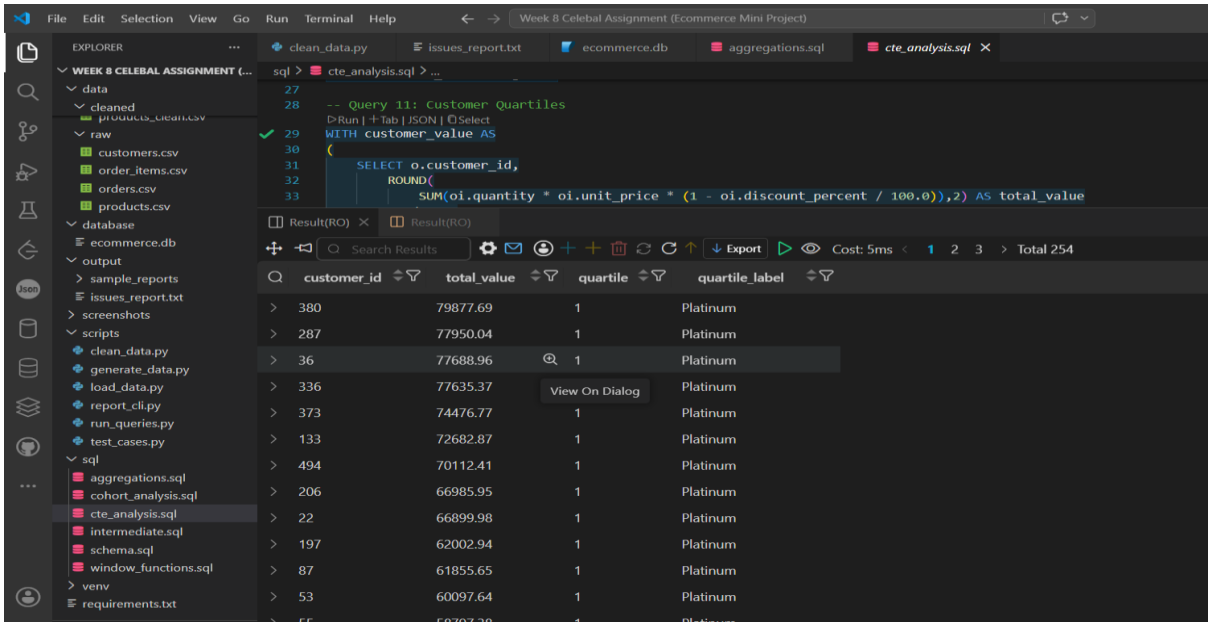
They were categorized into Platinum, Gold, Silver, and Bronze groups.

Screenshot 15: Step7_Customer_Segmentation



Submitted By: Jiya Singla(MMDU)

Screenshot 16: Step7_Quartile_Classification



Step 8: Command-Line Reporting Tool

A Python CLI tool was developed to generate reports directly from the SQLite database.

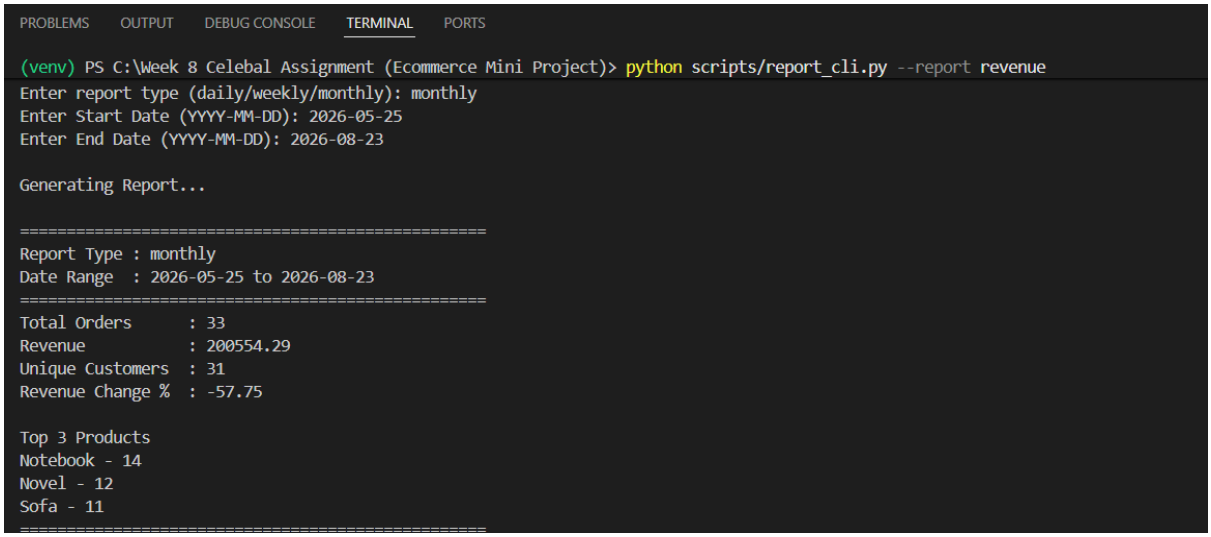
Example commands:

python report_cli.py --report revenue

python report_cli.py --report top_customers

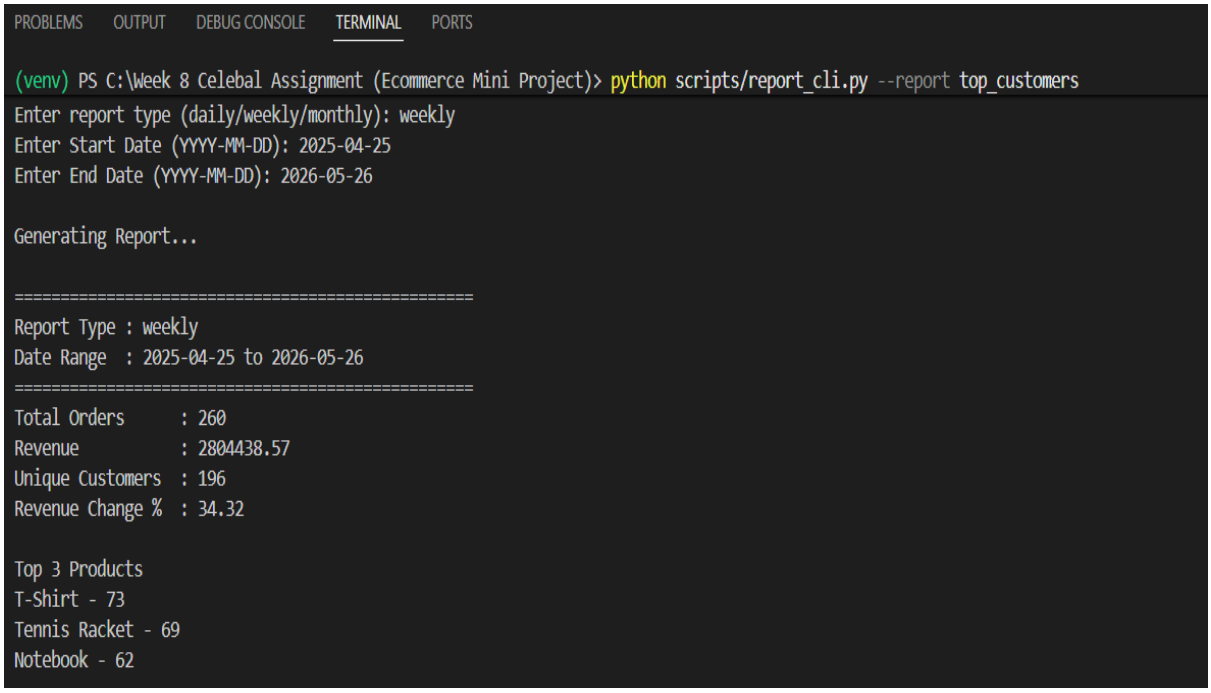
python report_cli.py --report retention

Screenshot 17: Step8_CLI_Execution

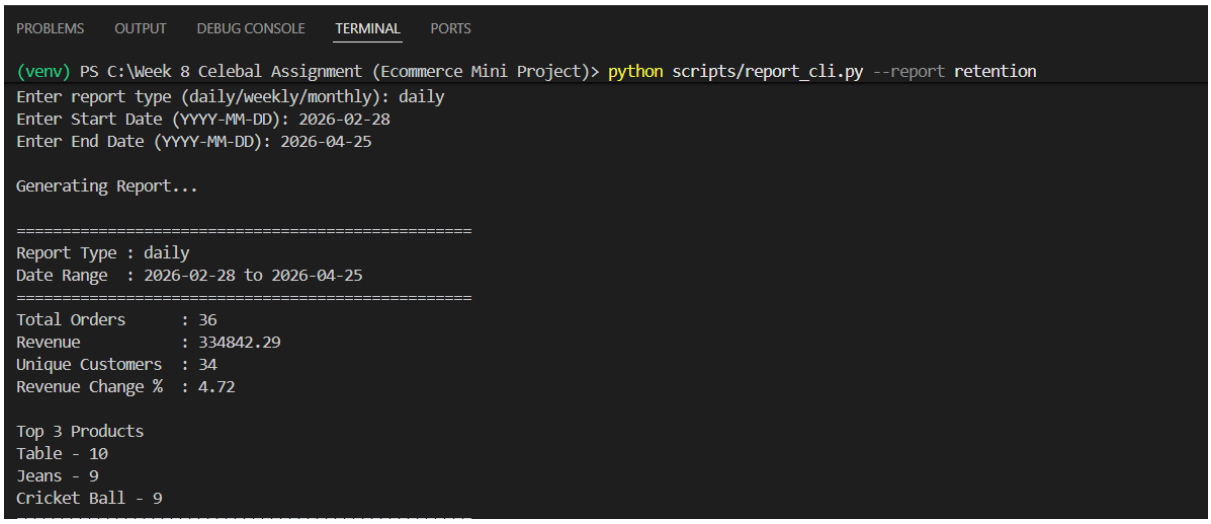


Submitted By: Jiya Singla(MMDU)

Screenshot 18: Step8_CLI_Top_Customers_Report



Screenshot 19: Step8_CLI_Retention_Report



Step 9: Edge Case Handling

The system was tested for different scenarios such as missing files, invalid report names, empty datasets, future dates, duplicate records, and database connection failures to ensure reliable execution.

Step 10: Documentation

Project documentation was completed by preparing a README.md file containing the project overview, folder structure, setup instructions, execution steps, and sample outputs. All source code, SQL scripts, datasets, and reports were organized according to the required project structure for submission.

6. CONCLUSION

The E-Commerce Order Analytics System successfully implements a complete data analytics pipeline from dataset generation to business reporting. Python was used for data generation, cleaning, and validation, while SQL was used to perform advanced analytics such as joins, aggregations, window functions, customer segmentation, cohort analysis, and retention analysis. The CLI reporting tool enables dynamic report generation, and robust validation ensures reliable data processing. This project demonstrates practical skills in Python, SQL, database management, and analytics.

Submitted By: Jiya Singla(MMDU)